IT4060
Machine Learning
Assignment 1

Name :- Keshan B K

Batch :- Y4.S1.WE.SE.01 Submission

Date:- 04/02/2026

1. INTRODUCTION

In the modern digital era, organizations generate massive volumes of data through customer interactions, transactions, and service usage. Extracting meaningful insights from this data is essential for improving decision-making and business performance. Machine Learning (ML) provides powerful techniques to analyze such data and build predictive models.

One of the most critical applications of machine learning in the business domain is **customer churn prediction**. Customer churn refers to the phenomenon where customers discontinue their relationship with a company. High churn rates can significantly impact revenue, customer lifetime value, and long-term sustainability.

From a business perspective, retaining existing customers is significantly more cost-effective than acquiring new ones. Therefore, accurately predicting churn enables organizations to identify high-risk customers and take proactive actions such as targeted marketing, personalized offers, and service improvements.

This project focuses on applying multiple supervised machine learning algorithms to predict customer churn using a real-world dataset and evaluating their performance using appropriate metrics.

2. Problem Statement

The main objective of this study is to develop predictive models that can accurately classify whether a customer is likely to churn or not.

This problem is formulated as a **binary classification problem**, where:

- Class **0** represents customers who continue using the service
- Class **1** represents customers who discontinue the service (churn)

The primary challenge in this problem is not only achieving high accuracy but also ensuring that churn customers are correctly identified. This is because misclassifying churn customers (false negatives) can lead to missed opportunities for retention and potential revenue loss.

Therefore, in addition to accuracy, evaluation metrics such as **recall and F1-score** are considered crucial in this study.

3. Dataset Description

The dataset used in this project is the Telco Customer Churn dataset, which is publicly available and widely used for machine learning applications.

3.1 Dataset Overview

- Total number of instances: 7043
- Number of rows : 21
- Data format: structured tabular dataset
- Domain: telecommunications industry.

3.2 Feature Description

The dataset consists of multiple features categorized as follows:

Demographic Features

- Gender
- Senior Citizen status
- Partner status
- Dependents

Account Information

- Tenure (number of months the customer has stayed)
- Contract type (monthly, yearly, etc.)
- Payment method

Service-related Features

- Internet service type
- Online security
- Tech support
- Streaming services

Financial Features

- Monthly charges
- Total charges

3.3 Target Variable

- **Churn** (Yes/No), which is converted into binary format:
    - Yes → 1
    - No → 0

3.4 Dataset Challenges

One of the key challenges in this dataset is **class imbalance**, where the number of non-churn customers is significantly higher than churn customers. This imbalance can lead to biased model predictions.

4. Data Preprocessing

Data preprocessing is a crucial step in machine learning, as raw data is often incomplete, inconsistent, or unsuitable for modeling.

4.1 Data Cleaning

- Converted TotalCharges to numeric format
- Removed missing or invalid values
- Dropped irrelevant columns (e.g., customer ID

4.2 Encoding Categorical Variables

Machine learning models require numerical input. Therefore:

- Categorical features were transformed using encoding techniques
- Label encoding was applied for simplicity

4.3 Feature Scaling

Feature scaling was applied using **StandardScaler** to normalize numerical features. This is particularly important for distance-based algorithms such as KNN, where feature magnitude affects model performance.

4.4 Train-Test Split

The dataset was divided into:

- Training set (used to train the model)
- Testing set (used to evaluate model performance)

This ensures that the model can generalize well to unseen data.

4.5 Class Imbalance Consideration

The imbalance in the dataset was identified as a major issue affecting model performance. This imbalance leads to models favoring the majority class (non-churn).

Future improvements can include:

- SMOTE (Synthetic Minority Oversampling)
- Class weighting technique

5. Methodology (Algorithms Used)

In this study, four supervised machine learning algorithms were applied and compared.

5.1 Logistic Regression

Logistic Regression is a linear classification algorithm that models the probability of a binary outcome using a logistic (sigmoid) function.

Advantages

- Simple and efficient
- Works well for linearly separable data
- Provides probabilistic outputs

Limitations

- Cannot capture complex non-linear relationships

5.2 Decision Tree

Decision Tree is a hierarchical model that splits data based on feature conditions.

Advantages

- Easy to interpret and visualize
- Handles both numerical and categorical data

Limitations

- Prone to overfitting
- Sensitive to small changes in data

5.3 Random Forest

Random Forest is an ensemble learning technique that combines multiple decision trees to improve performance.

Advantages

- Reduces overfitting
- High accuracy
- Robust to noise

Limitations

- Less interpretable compared to a single decision tree

5.4 K-Nearest Neighbors (KNN)

KNN is a non-parametric algorithm that classifies data points based on the majority class of their nearest neighbors.

Advantages

- Simple and intuitive
- No training phase

Limitations

- Computationally expensive
- Sensitive to feature scaling

6. Results

The models were evaluated using multiple performance metrics including accuracy, recall, and F1-score. Additionally, confusion matrices and classification reports were used to analyze performance in detail.

6.1 Accuracy Analysis

The accuracy results indicate that Random Forest achieved the highest accuracy among all models. This suggests that it is more effective in capturing overall patterns in the dataset.

6.2 Recall Analysis

Recall measures the ability of the model to correctly identify churn customers. The results show that recall values are relatively low across all models, indicating difficulty in detecting churn cases.

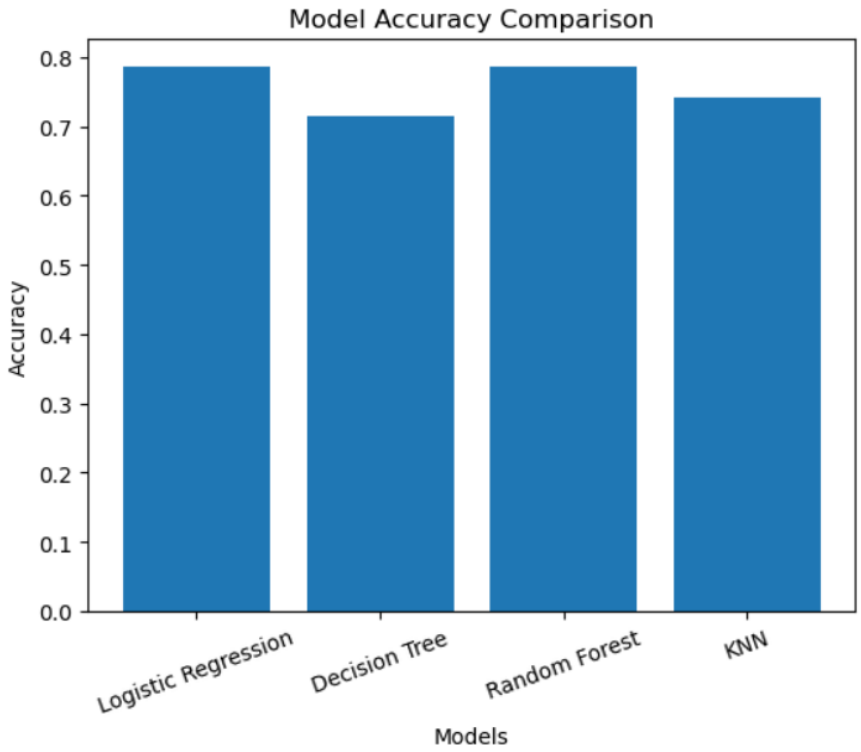
6.3 F1-Score Analysis

The F1-score provides a balance between precision and recall. Among the models, Random Forest achieved the most consistent F1-score, indicating better overall performance.

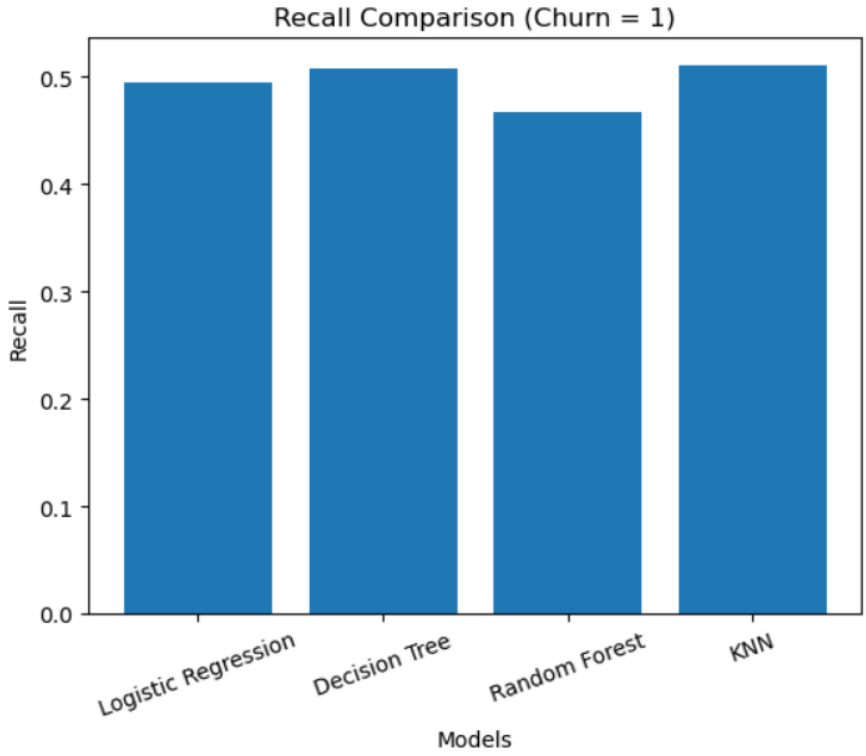
6.4 Numerical Results

Model	**Accuracy**	**Recall (Churn=1)**	**F1 Score**
Logistic Regression	0.78	0.49	0.55
Decision Tree	0.72	0.50	0.49
Random Forest	0.80	0.49	0.56
KNN	0.74	0.51	0.51

6.5 Accuracy Chart



6.6 Recall Accuracy Chart



7. Model Comparison

The comparison of models reveals that:

- **Random Forest** performs best in terms of accuracy and overall stability
- **Logistic Regression** provides reasonable performance but is limited in capturing complex patterns
- **Decision Tree** shows lower performance due to overfitting
- **KNN** provides moderate results but is sensitive to data scaling

Despite differences in accuracy, all models exhibit limitations in detecting churn customers.

8. Discussion

The results highlight several important observations:

8.1 Impact of Class Imbalance

The dataset imbalance significantly affects model performance. Models tend to predict the majority class, resulting in low recall for churn customers.

8.2 Importance of Evaluation Metrics

Accuracy alone is not sufficient for evaluating model performance in imbalanced datasets. Metrics such as recall and F1-score provide a more comprehensive evaluation.

8.3 Business Implications

From a business perspective, failing to identify churn customers can lead to revenue loss. Therefore, improving recall is more important than simply increasing accuracy.

8.4 Model Behavior

Random Forest performs better due to its ensemble nature, while simpler models struggle with complex relationships.

8.5 Limitations

- Class imbalance not handled
- No hyperparameter tuning
- Limited feature engineering

8.6 Future Improvements

- Apply SMOTE for balancing data
- Use GridSearchCV for tuning
- Add feature engineering
- Test additional models (SVM, XGBoost)

9. Conclusion

This study explored the application of multiple machine learning algorithms for predicting customer churn.

The results indicate that Random Forest is the most effective model in terms of overall performance. However, improvements are necessary to enhance churn detection.

Future work can include:

- Applying advanced resampling techniques (e.g., SMOTE)
- Hyperparameter tuning
- Feature engineering
- Testing additional machine learning models

This project demonstrates the importance of selecting appropriate models and evaluation metrics in solving real-world problems.

10. Appendix

Load Dataset

```
import pandas as pd

df = pd.read_csv("../data/Telco-Customer-Churn.csv")

df.head()
```

Explore Data

```
df.info()

df.describe()

df.isnull().sum()
```

Data Cleaning

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

df = df.dropna()

df = df.drop("customerID", axis=1)
```

Encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == 'object':

        df[col] = le.fit_transform(df[col])
```

Import libraries

```
import matplotlib.pyplot as plt

import seaborn as sns
```

Churn Distribution

```
sns.countplot(x='Churn', data=df)

plt.title("Churn Distribution")

plt.show()
```

Correlation Heatmap

```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()
```

Feature vs Target

```
sns.boxplot(x='Churn', y='tenure', data=df)
plt.title("Tenure vs Churn")
plt.show()
```

Monthly Charges vs Churn

```
sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
plt.show()
```

Split data

```
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

Train/Test split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

LOGISTIC REGRESSION

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
acc_lr = accuracy_score(y_test, y_pred_lr)
print ("Logistic Regression Accuracy:", acc_lr)
```

DECISION TREE

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
acc_dt = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", acc_dt)
```

RANDOM FOREST

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
acc_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", acc_rf)
```

KNN

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
acc_knn = accuracy_score(y_test, y_pred_knn)
print("KNN Accuracy:", acc_knn)
```

COMPARISON TABLE

```
import pandas as pd
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest", "KNN"],
    "Accuracy": [acc_lr, acc_dt, acc_rf, acc_knn]
})
```

Results

Classification Report

```
from sklearn.metrics import classification_report
print("Logistic Regression Report:\n")
print(classification_report(y_test, y_pred_lr))

print("Decision Tree Report:\n")
print(classification_report(y_test, y_pred_dt))

print("Random Forest Report:\n")
print(classification_report(y_test, y_pred_rf))

print("KNN Report:\n")
print(classification_report(y_test, y_pred_knn))
```

Model Accuracy Chart

```
import matplotlib.pyplot as plt
models = ["Logistic Regression", "Decision Tree", "Random Forest", "KNN"]
accuracies = [acc_lr, acc_dt, acc_rf, acc_knn]
plt.figure()
plt.bar(models, accuracies)
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
```

```
plt.xticks(rotation=20)
plt.show()
```

Racall Accuracy Chart

```
from sklearn.metrics import recall_score
recalls = [
    recall_score(y_test, y_pred_lr),
    recall_score(y_test, y_pred_dt),
    recall_score(y_test, y_pred_rf),
    recall_score(y_test, y_pred_knn)
]
plt.figure()
plt.bar(models, recalls)
plt.title("Recall Comparison (Churn = 1)")
plt.xlabel("Models")
plt.ylabel("Recall")
plt.xticks(rotation=20)
plt.show()
```

F1 Score Comparison

```
from sklearn.metrics import f1_score
f1_scores = [
    f1_score(y_test, y_pred_lr),
    f1_score(y_test, y_pred_dt),
    f1_score(y_test, y_pred_rf),
    f1_score(y_test, y_pred_knn)
]
plt.figure()
plt.bar(models, f1_scores)
plt.title("F1 Score Comparison")
plt.show()
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix

import seaborn as sns

import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt='d')

plt.title("Random Forest Confusion Matrix")

plt.show()
```